COFFEE CHAIN DATABASE

Redesign, Analytics and Performance Study

A MySQL 8 database for a coffee-chain ordering and membership system, modeled on Starbucks Thailand

Author: Sirirampa Luangphiphatsorn (6788057)

Faculty of Information and Communication Technology, Mahidol University

Based on: ITCS216 Fundamentals of Database Systems, group project (Semester 1/2025)

Original team: Sirirampa Luangphiphatsorn, Chanachon Ruanpheng, Yanapat Sittirat, Chanapron Sumkhemthong, Ratchanon Srithongkul

Version: 2.0, October 2026

Educational project, not affiliated with Starbucks. All customer, order and payment data in this project is synthetic.

Contents

Executive Summary 3
1. Introduction and Scope 4
1.1 Business domain 4
1.2 Project history 4
1.3 Objectives 4
1.4 Tools 4
2. Business Processes and Requirements 5
2.1 Business rules enforced by the database 5
3. Database Design (v2) 6
3.1 Entity-relationship diagram 6
3.2 Tables 7
3.3 Design decisions worth explaining 7
4. Improvements from v1 to v2 8
4.1 Problems found in the original files 8
5. Implementation and Test Data 9
5.1 Schema 9
5.2 Two data sets 9
5.3 Making generated data realistic 9
5.4 Verification 10
6. Business Analytics with Advanced SQL 11
6.1 Selected results 11
7. Business Rules: Triggers and Stored Procedures 13
7.1 Triggers 13
7.2 Stored procedures and function 13
7.3 Demonstration 13
8. Performance Tuning with Indexes 15
8.1 Method 15
8.2 Query plan before and after (MySQL 8.4) 15
8.3 Results 15
8.4 Indexes 16
8.5 Lessons learned 16
9. Problems Encountered and Solutions 18
10. Conclusion and Future Work 19
10.1 Conclusion 19
10.2 Limitations 19
10.3 Future work 19
Appendix A. Repository and How to Run 20
Run order 20
Credits 20

(Word asks to update fields when the file opens. Choose Yes to fill in this table. If it is still empty, right-click it and choose Update Field.)

Executive Summary

This project takes a database designed as a second-year group assignment and rebuilds it to a professional standard. The original version (v1) had 11 tables, about 120 hand-typed rows and 25 basic queries. Reviewing it showed several design problems. Some times were stored without a time of day. One order could not contain the same drink twice. Stored order totals did not match their items. Several business processes described in the report had no tables to support them.

Version 2 fixes these problems and extends the project in four directions:

- **Design:** 15 tables, 16 documented design changes, constraints that reject invalid data, and an ER diagram.
- **Data:** a repeatable pure-SQL generator that creates 200,018 orders for 20,010 members across 50 branches, verified by 13 integrity checks.
- **SQL:** 12 business questions answered with CTEs, window functions and a recursive CTE. Business rules are enforced by 9 triggers and 4 stored procedures, and order placement is a single transaction.
- **Performance:** an `EXPLAIN ANALYZE` before/after study of 6 purpose-built indexes. Looking up one day of orders went from 50.2 ms to 1.71 ms on MySQL 8.4 (200,020 to 492 rows read). The full analytics script went from 9.3 s to 6.4 s.

The report also records what did **not** work: an index that made queries slower, queries that no index can help, and the extra cost that indexes add to inserts. These findings are as important as the speed-ups.

Measure	v1 (group project)	v2 (this project)
Tables	11	15
Sample data	~120 rows	1.4 million rows across all tables
Queries	25 basic (WHERE, JOIN, GROUP BY)	12 analytical + 4 views + 8 benchmarks
Business rules	CHECK constraints only	9 triggers, 1 function, 4 procedures
Data verification	none	13 automated checks, all passing
Performance work	none	6 indexes, measured before/after

1. Introduction and Scope

1.1 Business domain

Starbucks is a global coffee chain that sells coffee, espresso drinks, teas, food and merchandise. In Thailand customers can order in the store, through the mobile app, or by delivery. Members collect points (Stars) and can pay with a stored-value Starbucks Card. The database supports this business: members, branches, the menu, orders and how they are fulfilled, payments, promotions and loyalty points.

1.2 Project history

The original database was built by a team of five in ITCS216 (Semester 1/2025) in two phases: database design (ERD and relational mapping) and implementation (MySQL tables, sample data and 25 queries, five per member). This report describes an individual redesign and extension of that work by Sirirampa Luangphiphatsorn. The original files are kept unchanged in the `v1-original` folder of the repository for comparison.

1.3 Objectives

1. Find and fix the design flaws in the v1 schema, and record the reason for every change.
2. Produce enough realistic data to make analysis and performance testing meaningful.
3. Answer real business questions using advanced SQL.
4. Move important business rules into the database so that no application can break them.
5. Measure, rather than assume, the effect of indexing.

1.4 Tools

Tool	Used for
MySQL 8.0 / 8.4	Database server (InnoDB). CTEs, window functions, JSON_TABLE and EXPLAIN ANALYZE all need MySQL 8
MySQL Workbench	Running scripts and reading query plans
dbdiagram.io (DBML)	ER diagram
Git and GitHub	Version control. Each phase is a separate commit
Python (optional)	Automated benchmark timing (`benchmark.py`)

2. Business Processes and Requirements

The business processes come from the original Phase I report. The right-hand column shows where each one is implemented in v2. Several were described in v1 but had no tables behind them.

Process	Description	Implemented in v2 by
Membership	Customers register with name, email, date of birth and phone. Members collect points from purchases and redeem them for rewards. Levels are Green and Gold.	Member, Member_Address, Point_Transaction, `sp_redeem_points`, Gold upgrade trigger
Ordering	Three channels: In-store Pickup (choose a branch), Order to Table (served at the table), and Delivery (to the customer's address). Drinks can be customised (size, milk, extra shot).	Orders + subtypes InStorePickup / OrderToTable / Delivery, Order_Item, `sp_place_order`
Payment	Members pay by Starbucks Card or credit card. A member can hold several Starbucks Cards. After payment the order status changes.	Payment, Starbucks_Card, card-balance and status triggers
Promotion	Seasonal, member-only and point-multiplier promotions, offered on specific products at specific branches.	Promotion, Offer, Orders.Promotion_ID, view `v_promotion_status`, `fn_calc_discount`
Product management	Maintain the menu, availability and prices, and analyse sales performance.	Product (Base_Price, Is_Active, Is_Seasonal), analytics Q4, Q10

2.1 Business rules enforced by the database

These rules are checked inside MySQL, so they hold no matter which application inserts the data:

- An order total always equals the sum of its items.
- Discontinued products cannot be ordered, and items can only be added while an order is Pending.
- Drive Thru branches cannot take Order-to-Table orders, and orders must fall within a branch's opening hours.
- A promotion can only be used between its start and end dates.
- Delivery orders cannot be paid in cash.
- A Starbucks Card can only be used by its owner, must be Active, and must have enough balance.
- Points can never go below zero, and members are upgraded to Gold at 500 points.
- Cancelled and Completed orders are final. A Paid order cannot return to Pending.

3. Database Design (v2)

3.1 Entity-relationship diagram

Figure 1 shows the 15 tables of the v2 schema. Orders sits at the centre. Each order belongs to one member and one branch, can use one promotion, has one or more items, exactly one fulfilment subtype, and at most one receipt and payment.

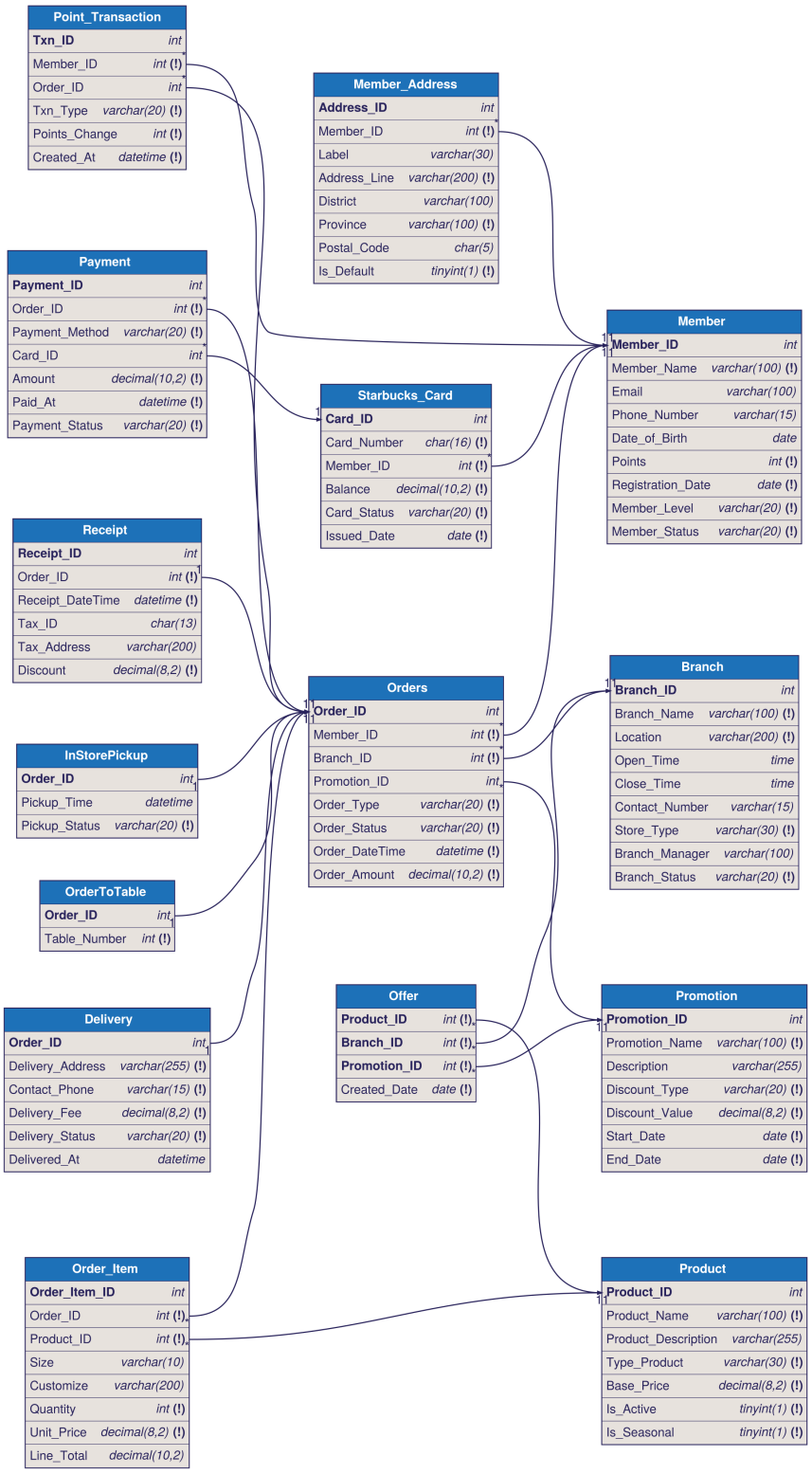


Figure 1. ER diagram of schema v2 (generated from docs/erd_v2.dbml)

3.2 Tables

Table	Purpose	Key points
Member	Registered customers	UNIQUE email and phone, CHECK on level, status and points ≥ 0
Member_Address	Saved delivery addresses (many per member)	Surrogate Address_ID. Address split into line, district, province, postcode
Product	Menu items and merchandise	Base_Price, Is_Active, Is_Seasonal, type CHECK
Branch	Stores	Open_Time and Close_Time as TIME. Store type and status CHECKs
Promotion	Campaigns	Discount_Type (Percent / Baht / Point_Multiplier) + Discount_Value. Status is derived, not stored
Offer	Which promotion runs on which product at which branch	Ternary relationship, composite primary key
Starbucks_Card	Stored-value cards	UNIQUE card number. Balance ≥ 0
Orders	Order header	DATETIME, type and status CHECKs, optional Promotion_ID. Order_Amount kept by triggers
Order_Item	Order lines (was "Contain")	Own surrogate key. Unit_Price at time of sale. Line_Total is a generated column
InStorePickup / OrderToTable / Delivery	Fulfilment subtypes (one per order)	PK = FK to Orders. Delivery stores an address snapshot
Receipt	Tax receipt	One per order (UNIQUE Order_ID)
Payment	Payments and refunds	CHECK: a card is required if and only if the method is StarbucksCard
Point_Transaction	Every change to a member's points	Earn / Redeem / Adjust / Expire. Member.Points is the running total

3.3 Design decisions worth explaining

Supertype/subtype for order channels. The three channels need different data (pickup time, table number, delivery address), so each has its own table that shares the order's primary key. This avoids one wide Orders table full of NULLs.

Price at time of sale. Product.Base_Price is the current menu price, and Order_Item.Unit_Price records what the customer actually paid. Without the second column, a price change would silently rewrite the history of past orders.

Address snapshot on delivery. Delivery copies the address text instead of referencing Member_Address. If a member later edits or deletes an address, past deliveries still show where the order actually went.

Two deliberate cached totals. Orders.Order_Amount and Member.Points could be calculated every time, but they are read on almost every screen, so they are stored. This is a controlled denormalisation, and triggers guarantee the stored value always matches its source rows. The verification checks confirm it.

Derived status instead of stored status. v1 stored Promotion_Status by hand, and two promotions that had already ended were still marked Active. In v2 the status is calculated from the dates in the view `v_promotion_status`, so it can never be wrong.

4. Improvements from v1 to v2

Every change was recorded with its reason (`docs/schema-v2-decisions.xlsx`). Items 1–16 were implemented. Items 17–19 were deliberately left out to keep the scope under control, and are listed as future work in Chapter 10.

#	v1	v2	Reason
1	Order and pickup times stored as DATE	DATETIME	No time of day means no peak-hour analysis
2	Contain PK = (Product_ID, Order_ID)	Order_Item with its own Order_Item_ID	Same drink twice with different milk was impossible
3	Product had no price	Base_Price + Unit_Price on each line	Current price vs. price at time of sale
4	No discount value	Discount_Value	"Percent" without the percentage was meaningless
5	Order_Amount typed in	Maintained by triggers	Order 400001 stored 120 but its items totalled 380
6	Promotion_Status typed in	Calculated in a view	Ended promotions still showed "Active"
7	Email and phone could repeat	UNIQUE	Two members cannot share an email
8	No Delivery channel	Delivery table + order type	Required by the business process
9	No card table	Starbucks_Card	Members can hold several cards
10	No payment record	Payment	How was each order paid?
11	Points were only a total	Point_Transaction history	Auditable, and lets triggers maintain the total
12	Receipt discount had no source	Orders.Promotion_ID	Promotion performance can be measured
13	IDs typed by hand	AUTO_INCREMENT	Standard practice, needed for bulk data
14	Table named `Order`	`Orders`	ORDER is a reserved word
15	Address text was part of the PK	Address_ID + split fields	Long, editable text is a poor key
16	Opening hours as text	Open_Time / Close_Time (TIME)	Allows "is this branch open now?" checks

4.1 Problems found in the original files

Problem	Where	Fixed by
Stored order total did not match its items (120 vs 380)	Order 400001	Item 5, triggers
Ended promotions still marked Active	Promotions 300003, 300005	Item 6, view
Order-to-Table order placed at a Drive Thru branch	Order 400008	Data fixed, rule added in `sp_place_order`
A tax ID with 12 digits instead of 13	Receipt 500010	Tax_ID is now CHAR(13)
Query referenced aliases `m.` and `o.` without a JOIN	v1 query file	Rewritten in v2 analytics
Typo `Poins`, comparing a date to `CURDATE()` with `=`, and GROUP BY not valid under ONLY_FULL_GROUP_BY	v1 query file	Rewritten in v2 analytics

5. Implementation and Test Data

5.1 Schema

`schema/schema_v2.sql` creates the database, all 15 tables with named constraints, the view `v_promotion_status`, and three triggers that keep Orders.Order_Amount equal to the sum of Order_Item.Line_Total after every insert, update or delete of an item. Line_Total is a stored generated column (`Quantity * Unit_Price`), so it cannot disagree with its inputs.

```
CREATE TRIGGER trg_order_item_ai AFTER INSERT ON Order_Item FOR EACH ROW
BEGIN
    UPDATE Orders
    SET Order_Amount = (SELECT COALESCE(SUM(Line_Total), 0)
                        FROM Order_Item WHERE Order_ID = NEW.Order_ID)
    WHERE Order_ID = NEW.Order_ID;
END
```

5.2 Two data sets

`seed/seed_small.sql` is the v1 sample data, migrated to the new schema and corrected, so it is small enough to read. `seed/seed_large.sql` generates realistic volume for analysis and performance testing. It is pure SQL, needs no extra tools, and produces exactly the same data on every run.

Table	Rows	Table	Rows
Member	20,010	Orders	200,018
Member_Address	20,011	Order_Item	378,720
Branch	50	InStorePickup	110,097
Product	40	OrderToTable	50,123
Promotion	21	Delivery	39,798
Starbucks_Card	7,006	Receipt / Payment	190,098 each
		Point_Transaction	190,109

5.3 Making generated data realistic

Random data is easy. Data that behaves like a real business takes rules. The generator applies these:

- Older members order more often (loyal customers), and no one orders before their registration date.
- Order volume grows over time (about 7,000 orders a month in early 2025, about 14,000 by September 2026).
- Peak hours at 08:00, 10:00 (when malls open), 12:00 and 17:00, and every order falls within its branch's opening hours.
- A few products dominate sales (Caffè Latte first). Merchandise sells rarely.
- Drive Thru branches never get table orders, delivery is never paid in cash, and cancelled orders have no payment.
- Payment = items − discount + delivery fee. Points = 1 per 10 THB, doubled during point-multiplier promotions.
- Simulated assumption: 60% of promotion orders add one extra item. This is a rule chosen for the simulation, not an observation, and it explains the promotion uplift reported in Chapter 6.

Randomness comes from MD5 hashes of row numbers rather than `RAND(seed)`. While testing, nearby seeds in MySQL produced correlated number streams, so the "independent" random columns were not actually independent.

5.4 Verification

`seed/verify_data.sql` runs 13 checks, and each must return 0 problems. All 13 pass on the full data set, including after the demo orders from Chapter 7.

Check (each must be 0)	Check (each must be 0)
Order_Amount ≠ sum of items	Order-to-Table at a Drive Thru
Order without items	Order outside branch opening hours
Order missing its subtype row	Delivery paid by cash
Subtype row does not match Order_Type	Card payment with another member's card
Payment ≠ items − discount + delivery fee	Promotion used outside its dates
Cancelled order still holding a successful payment	Member.Points ≠ sum of Point_Transaction
Order placed before the member registered	

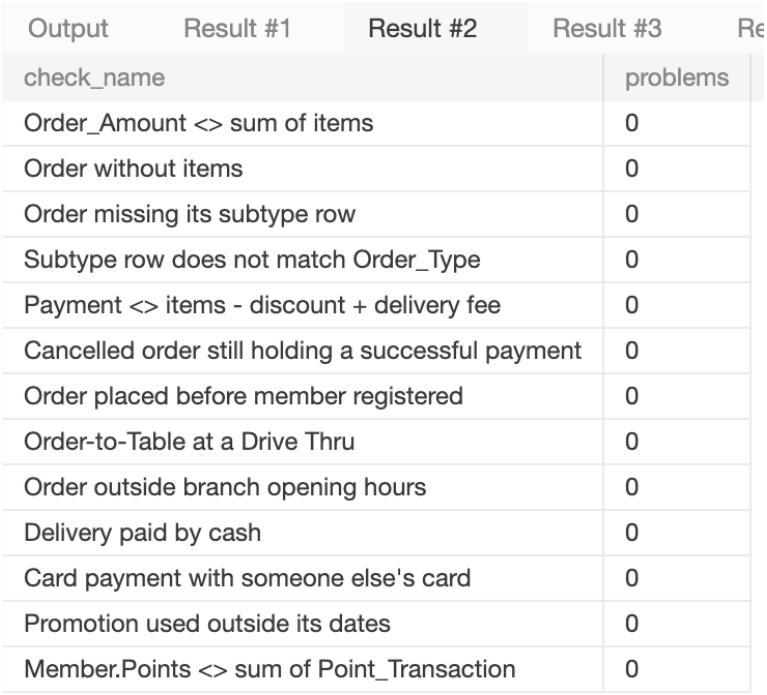


Figure 2. verify_data.sql in MySQL Workbench (MySQL 8.4.11): all 13 checks return 0

6. Business Analytics with Advanced SQL

`queries/02_analytics.sql` answers 12 business questions. To keep results repeatable, "today" is fixed at 30 September 2026, the last day of the data, and every query ignores rows after that date. `queries/01_views.sql` provides four reusable views (order summary, daily branch sales, member summary, product sales).

#	Business question	SQL technique
Q1	How is revenue growing month by month? Year-to-date total?	CTE, LAG, SUM() OVER (PARTITION BY year)
Q2	What is the underlying trend in daily orders?	7-day moving average (window frame)
Q3	Which branches earn the most? Is it 80/20?	RANK, running SUM() OVER
Q4	Top 3 products in each branch	DENSE_RANK() OVER (PARTITION BY)
Q5	When is each branch busiest?	ROW_NUMBER, conditional aggregation pivot
Q6	Who are the best, loyal and slipping customers?	Chained CTEs, NTILE (RFM)
Q7	Which regular customers have gone quiet?	LAG on dates, per-customer averages
Q8	Do new customers come back?	Cohort retention, MIN() OVER, pivot
Q9	Did promotions raise spending, and at what cost?	CTE comparison, NULL-safe equality <=>
Q10	Which products are bought together?	Self-join
Q11	Median delivery time per branch	ROW_NUMBER + COUNT() OVER (median)
Q12	Unusually quiet days per branch	Recursive CTE calendar, CROSS / LEFT JOIN

6.1 Selected results

Q1: Revenue growth

Revenue in September 2026 was **5.26 million THB**, up **74%** on September 2025 (3.02 million). Year-to-date revenue at the end of September 2026 was 38.6 million THB.

Month	Paid orders	Revenue (THB)	Growth vs previous month
2026-05	10,519	4,121,401.50	+4.1%
2026-06	11,101	4,293,264.00	+4.2%
2026-07	12,145	4,751,792.50	+10.7%
2026-08	13,389	5,167,036.00	+8.7%
2026-09	13,708	5,261,892.25	+1.8%

Q3: Branch concentration

The top branch (Siam Paragon) earns 6.2% of revenue, and the top 5 together earn 19.1%. It takes **37 of the 50 branches** to reach 80% of revenue, so revenue is spread widely and the 80/20 pattern does not apply. The chain does not depend on a few flagship stores.

Q6: RFM customer segments

Segment	Customers	Avg days since last order	Avg orders	Share of revenue
Champions	3,062	11	16.6	27.5%
Loyal	4,835	25	10.7	26.3%
New / Promising	2,663	11	5.6	7.8%
At Risk	4,104	100	10.9	23.7%
Hibernating	3,898	136	5.2	10.7%

Segment	Customers	Avg days since last order	Avg orders	Share of revenue
Need Attention	1,441	37	5.5	4.1%

The "At Risk" group is the most actionable finding. These 4,104 customers used to order often (10.9 orders on average) and still account for 23.7% of revenue, but they have not ordered for about 100 days. Q7 turns this into a named list of regular customers whose current silence is more than three times their usual gap between visits.

Q10: Products bought together

Drink	Food	Orders together
Caffè Latte	Butter Croissant	1,433
Caffè Latte	Chocolate Chunk Cookie	1,138
Caffè Latte	Ham & Cheese Croissant	1,117
Caffè Latte	Blueberry Muffin	1,100
Caffè Latte	Tuna Sandwich	1,071

Latte with Butter Croissant is the natural candidate for a combo set.

Other results

- **Q5:** Mall branches peak at 10:00–11:00 (around 40% of their orders), when the malls open. Office and street branches peak at 08:00–09:00. On weekdays, 41% of orders are placed before 11:00.
- **Q9:** Orders using a promotion have baskets about 30% larger (for example 490 vs 377 THB for "Summer 2026 20 Baht"). This follows from the simulated rule in Section 5.3, so it shows that the query works rather than proving that promotions work.
- **Q11:** Median delivery time is about 43 minutes at every branch, with almost no difference between branches.
- **Q12:** Nine branches had at least one day in September 2026 with fewer than 3 orders. Drive Thru Hua Hin had the most, with 3 such days.

Note on interpretation. The data is synthetic, so trends such as growth and promotion uplift come from rules in the generator. The value of this chapter is that each question can be answered correctly and efficiently in SQL. The results are not claims about real Starbucks customers.

7. Business Rules: Triggers and Stored Procedures

7.1 Triggers

Trigger	When	What it does
trg_order_item_ai / au / ad	After an item is inserted, updated or deleted	Recalculates Orders.Order_Amount
trg_order_item_bi	Before an item is inserted	Blocks discontinued products and changes to non-Pending orders. Sets the size and fills Unit_Price from the menu
trg_orders_bu	Before an order is updated	Status only moves forward. Cancelled and Completed are final
trg_payment_bi	Before a payment is inserted	Starbucks Card must belong to the member, be Active, and have enough balance
trg_payment_ai	After a payment is inserted	Charges the card, awards points (×2 during promotions), sets the order to Paid
trg_payment_au	After a payment is refunded	Returns money to the card and takes back the points earned
trg_point_txn_ai	After a point transaction	Updates Member.Points and upgrades the member to Gold at 500 points

7.2 Stored procedures and function

`sp_place_order` creates a complete order (order, items, subtype, receipt and payment) in **one transaction**. Items arrive as a JSON array and are turned into rows with `JSON_TABLE`. If any step fails, an exit handler rolls everything back and passes the error message to the caller.

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    RESIGNAL;   -- pass the original error message to the caller
END;
...
INSERT INTO Order_Item (Order_ID, Product_ID, Size, Customize, Quantity, Unit_Price)
SELECT p_order_id, j.product_id, j.size, j.customize, j.qty, NULL
FROM JSON_TABLE(p_items, '$[*]' COLUMNS (
        product_id INT PATH '$.product_id',
        qty        INT PATH '$.qty' DEFAULT '1' ON EMPTY,
        size VARCHAR(10) PATH '$.size', customize VARCHAR(200) PATH '$.customize')) AS j;
```

Routine	Purpose
fn_calc_discount(promotion, amount)	Discount in THB (Percent, Baht, or 0 for point promotions)
sp_place_order(...)	Validated, all-or-nothing order creation
sp_cancel_order(order_id)	Cancels Pending or Paid orders. Triggers refund the card and take back points
sp_redeem_points(member, points)	Spends points with clear error messages
sp_branch_report(branch, from, to)	Three result sets for a manager: KPIs, top 5 products, orders by hour

7.3 Demonstration

`queries/05_demo.sql` places two valid orders, tries 11 invalid actions, redeems points and cancels an order. Every invalid action was blocked with a clear reason, and a blocked order left no rows behind, which shows the transaction was rolled back.

Attempted action	Result	Reason given
Order a discontinued product	BLOCKED	This product is no longer available
Order-to-Table at a Drive Thru	BLOCKED	Drive Thru branches have no tables

Attempted action	Result	Reason given
Order at 05:00 before opening	BLOCKED	Order time is outside the branch opening hours
Use an ended promotion	BLOCKED	Promotion is not valid on the order date
Pay for delivery in cash	BLOCKED	Delivery orders cannot be paid in cash
Pay with another member's card	BLOCKED	This Starbucks Card belongs to another member
Card balance too low	BLOCKED	Not enough balance on the Starbucks Card
Banned member places an order	BLOCKED	Member account is not active
Redeem more points than owned	BLOCKED	Not enough points
Re-open a Completed order	BLOCKED	Cancelled or Completed orders cannot change status
Cancel a Completed order	BLOCKED	Only Pending or Paid orders can be cancelled

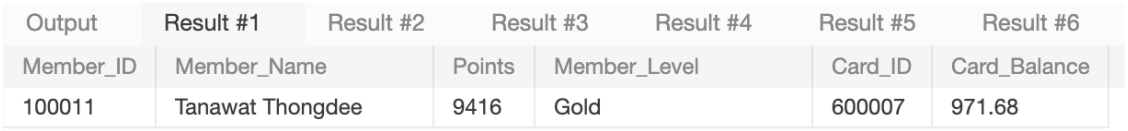


Figure 3. 05_demo.sql, first result: member 100011 before the demo (9,416 points, card balance 971.68)

In the valid flow, member 100011 paid 350 THB by Starbucks Card. The card balance fell from 971.68 to 621.68 and 35 points were added. After cancellation the balance returned to 971.68, the points went back to 9,416, and the point history shows both the Earn (+35) and the Adjust (−35) entries.

8. Performance Tuning with Indexes

8.1 Method

Eight realistic queries (`performance/01_benchmark_queries.sql`) were measured without the new indexes, then the indexes were created (`02_create_indexes.sql`) and the queries were measured again. Each query ran 15 times and the median is reported. Time depends on the machine, so the number of rows the storage engine actually read (from InnoDB handler counters) is reported too. That number does not depend on the machine.

8.2 Query plan before and after (MySQL 8.4)

Query B1, total sales for one day, captured in MySQL Workbench:

```
EXPLAIN
-> Aggregate: count(0), sum(orders.Order_Amount)  (cost=22379 rows=1) (actual time=50.2..50.2 rows=1 loops=1)
    -> Filter: ((orders.Order_DateTime >= TIMESTAMP'2026-09-30 00:00:00') and (orders.Order_DateTime < TIMESTAMP'2026-10-01 00:00:00'))
(cost=20164 rows=22159) (actual time=0.189..50.1 rows=492 loops=1)
        -> Table scan on Orders  (cost=20164 rows=199473) (actual time=0.132..41.3 rows=200020 loops=1)
```

Figure 4. Before: Table scan on Orders. 200,020 rows read, 50.2 ms

```
EXPLAIN
-> Aggregate: count(0), sum(orders.Order_Amount)  (cost=271 rows=1) (actual time=1.71..1.71 rows=1 loops=1)
    -> Index range scan on Orders using idx_orders_datetime over ('2026-09-30 00:00:00' <= Order_DateTime < '2026-10-01 00:00:00'), with
index condition: ((orders.Order_DateTime >= TIMESTAMP'2026-09-30 00:00:00') and (orders.Order_DateTime < TIMESTAMP'2026-10-01
00:00:00'))  (cost=222 rows=492) (actual time=0.208..1.63 rows=492 loops=1)
```

Figure 5. After: Index range scan using idx_orders_datetime. 492 rows read, 1.71 ms (about 29× faster)

8.3 Results

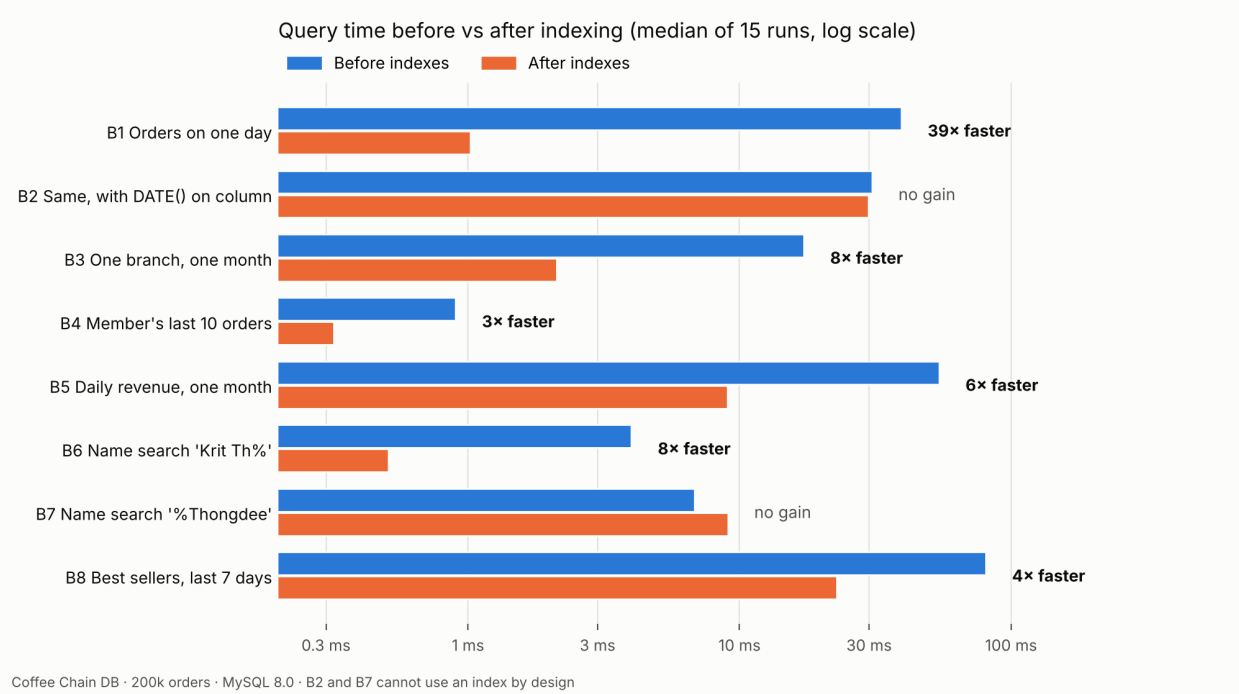


Figure 6. Median query time before and after indexing (log scale, MySQL 8.0 test environment)

Query	Before	After	Speed-up	Rows read
B1 Orders on one day	39.4 ms	1.0 ms	39×	200,022 → 492
B2 Same, written with DATE(column)	30.8 ms	29.8 ms	none	200,022 → 200,022
B3 One branch, one month	17.3 ms	2.1 ms	8×	13,142 → 1,779
B4 Member's last 10 orders	0.9 ms	0.3 ms	3×	261 → 9

Query	Before	After	Speed-up	Rows read
B5 Daily revenue, one month	54.4 ms	9.0 ms	6×	203,841 → 27,447
B6 Name search 'Krit Th%'	4.0 ms	0.5 ms	8×	20,012 → 30
B7 Name search '%Thongdee'	6.8 ms	9.1 ms	none	20,012 → 20,012
B8 Best sellers, last 7 days	80.6 ms	22.8 ms	4×	221,160 → 24,428

The full analytics script from Chapter 6 went from **9.3 s to 6.4 s**. The largest single gain was Q10 (products bought together), from 3.2 s to 1.1 s, thanks to a covering index on Order_Item.

8.4 Indexes

Index	Columns	Why
idx_orders_datetime	Order_DateTime	Most reports filter by a date range
idx_orders_branch_datetime	Branch_ID, Order_DateTime	Equality column first, range column last
idx_orders_member_datetime	Member_ID, Order_DateTime	Returns a member's orders already sorted, so no sort step
idx_payment_paidat	Paid_At, Payment_Status, Amount	Covering index: the query never reads the table
idx_order_item_order_product	Order_ID, Product_ID, Quantity	Covering index for the item self-join
idx_member_name	Member_Name	Prefix search on names

The single-column foreign-key indexes on Orders.Branch_ID, Orders.Member_ID and Order_Item.Order_ID became redundant, because the new composite indexes start with the same columns, and were removed.

8.5 Lessons learned

1. **A function on a column switches the index off (B2).** `DATE(Order_DateTime) = '2026-09-30'` and a range on `Order_DateTime` ask the same question, but only the range can use the index. The range was 39× faster, the function version gained nothing.
2. **A leading wildcard cannot use a B-tree index (B7).** An index is sorted from the first character, like a phone book. Searching "contains" needs a FULLTEXT index or a search engine.
3. **Column order matters.** With equality columns first and range columns last, (Branch_ID, Order_DateTime) cut B3 from 13,142 rows read to 1,779.
4. **Don't lead an index with a column that has almost no variety.** The first design was (Payment_Status, Paid_At, Amount). 99.99% of payments are "Success", but the optimizer assumed half the rows would match. It then started Q3 from Payment and made 190,000 random lookups into Orders, which made Q3 2× slower (408 → 904 ms). Moving Paid_At to the front fixed it.
5. **MySQL removes redundant foreign-key indexes by itself.** Once a composite index starting with Branch_ID existed, MySQL silently dropped the implicit FK_Orders_Branch index. The index script checks for this before dropping anything.
6. **Rewriting the query can beat any index.** Q12 originally checked NOT EXISTS separately for each of 1,500 branch-days and took about 1.4 s even with indexes. Summarising the month's orders once and then joining took about 33 ms, roughly 40× faster.
7. **Indexes are not free.** Inserting 20,000 orders took 278 ms before and 790 ms after (about 2.8× slower), and Payment's secondary indexes doubled in size (6.0 → 12.6 MB). A coffee chain reads far more than it writes, so the trade-off is worth it here.
8. **The optimizer is not always right.** Q1 (revenue for all months) got slower, 150 → 220 ms, because MySQL chose to scan the whole new index instead of the table. Indexes help when a query needs

a small part of a table. For whole-table reports, a summary table refreshed each night is the better tool.

9. Problems Encountered and Solutions

These problems came up during the project. Each one was found by testing, not by assumption.

Problem	Cause	Solution
Almost all generated orders landed at 20:00–21:00	`ELT()` returns text, so `GREATEST('7', 10)` compared strings, and '7' is greater than '10'	Convert to a number first: `0 + ELT(...)`
"Independent" random columns were correlated	Nearby `RAND(seed)` values in MySQL give similar streams	Hash-based randomness: MD5 of the row number
Almost every promotion order was given the same promotion	A promotion running all of 2025 overlapped the others, and a multi-row UPDATE JOIN picks any match	Pick one overlapping promotion explicitly with `ROW_NUMBER()`
Q9 lost its "without promotion" baseline	`NULL = x` returns NULL, not false	NULL-safe equality `<=>`
Seed file failed in Workbench	Safe-update mode blocks UPDATE statements without a key	`SET SQL_SAFE_UPDATES = 0` in the seed scripts
"Caffè" stored as "CaffÃ¨"	Client connection used latin1	`SET NAMES utf8mb4` in every script
Q12 returned nothing	No branch ever had a zero-sales day	Changed the question to "quiet days" (fewer than 3 orders)
Demo orders appeared in the analytics as an extra month	Queries had no upper date limit	Cap every query at the fixed end date `@as_of`
A new index made two queries 2× slower	Index led by a low-variety column	Reordered the columns (Section 8.5, lesson 4)
DROP INDEX failed	MySQL had already removed the index automatically	Check for the index before dropping it

10. Conclusion and Future Work

10.1 Conclusion

The redesigned database fixes every design problem found in the original, supports all five business processes the original report described, and enforces the business rules inside MySQL, where no application can bypass them. Realistic data at the scale of 200,000 orders made it possible to answer real business questions and to measure performance honestly. Six indexes made typical application and dashboard queries 3–39× faster. The project also showed where indexing does not help, and when rewriting a query is the better fix.

10.2 Limitations

- All data is synthetic. Business patterns reflect the generator's rules.
- Timings come from a single machine with default MySQL settings. The rows-read counts are the more reliable comparison.
- Member level is upgraded automatically but never downgraded, and points never expire.

10.3 Future work

- Normalise drink customisations into an option table (decision item 17).
- Add an Employee table instead of storing the branch manager as text (item 18).
- Split member names into first and last name (item 19).
- Nightly summary tables for whole-table reports such as Q1 and Q6 (Section 8.5, lesson 8).
- A FULLTEXT index for "contains" name search (Section 8.5, lesson 2).
- A small dashboard built on the reporting views.

Appendix A. Repository and How to Run

Repository: github.com/Pingxi57/coffee-chain-database

Folder	Contents
schema/	schema_v2.sql: tables, constraints, view, order-amount triggers
seed/	seed_small.sql, seed_large.sql (generator), verify_data.sql (13 checks)
queries/	01_views, 02_analytics, 03_triggers, 04_procedures, 05_demo
performance/	01_benchmark_queries, 02_create_indexes, 03_drop_indexes, benchmark.py, results.csv
docs/	ERD (DBML/PNG), v1→v2 decisions (xlsx), performance write-up and chart, this report
v1-original/	The original group-project SQL, unchanged

Run order

1. schema/schema_v2.sql
2. seed/seed_small.sql, then seed/seed_large.sql (about 1–3 minutes)
3. queries/01_views.sql, then 02_analytics.sql
4. queries/03_triggers.sql, 04_procedures.sql, 05_demo.sql (triggers must come after the seed data)
5. seed/verify_data.sql: every check should return 0
6. performance/01_benchmark_queries.sql, then 02_create_indexes.sql, then 01 again

Credits

Original ITCS216 group project (Semester 1/2025), supervised by Asst. Prof. Dr. Siripen Pongpaichet: Sirirampa Luangphiphatsorn, Chanachon Ruanpheng, Yanapat Sittirat, Chanapron Sumkhemthong, Ratchanon Srithongkul. Version 2 redesign, data generator, analytics, business rules, performance study and this report by Sirirampa Luangphiphatsorn.